

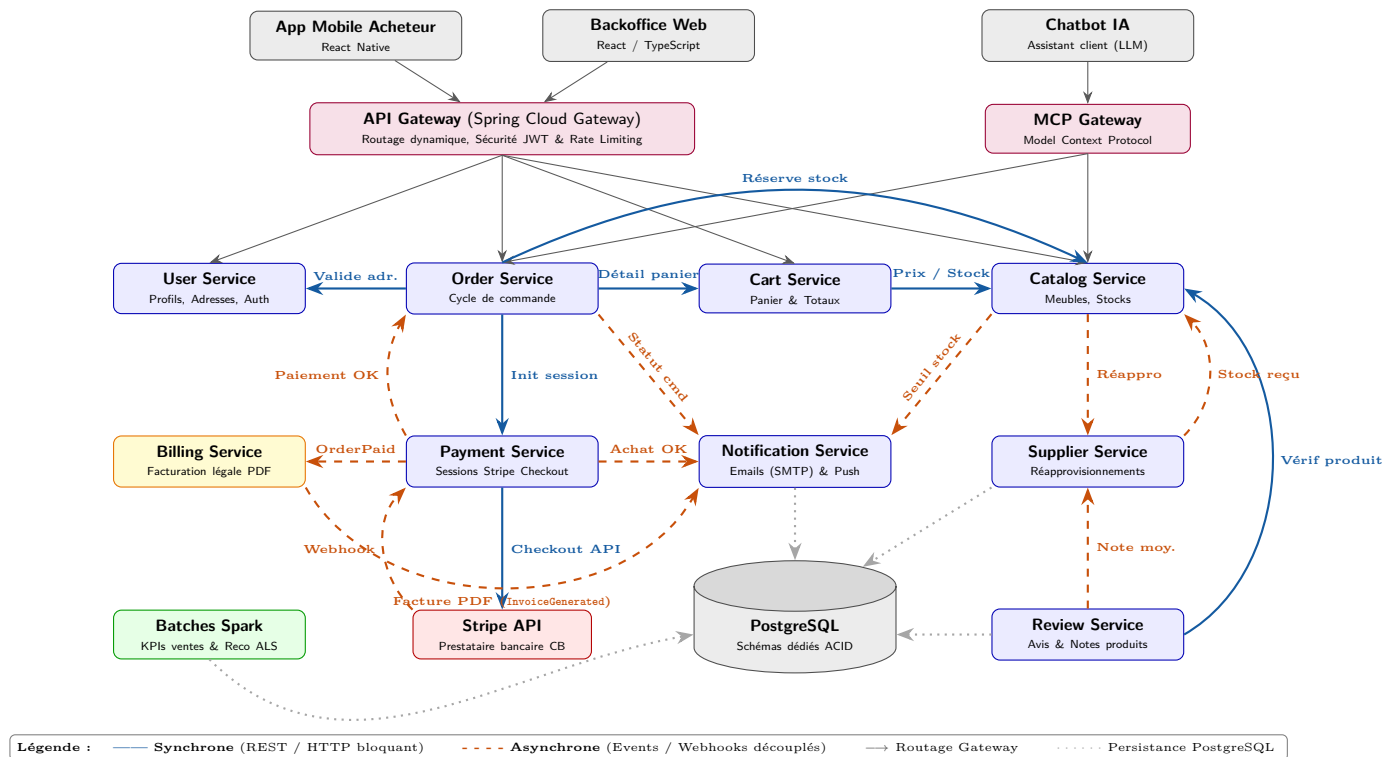
GOLDEN HOUSE

Architecture des microservices

Auteurs : Justin Chapon, Nabil Bennacer, Tom Cantillon, Salma El Annaby, Manuela Zapata Quirós

Polytech Montpellier – IG5
Module : IWA 2
Semestre 9 – Septembre 2026

1. Schéma des microservices



2. Synchrone et Asynchrone

- **Synchrone (REST / HTTP)** : Requêtes bloquantes exigeant une réponse immédiate pour finaliser l'action courante (lecture catalogue, calcul panier, validation adresse, réservation stock, session Stripe).
- **Asynchrone (Events / Webhooks)** : Traitements découplés dans le temps et tolérants aux pannes sans bloquer l'acheteur (webhook Stripe, confirmation paiement, génération facture PDF, notifications email/push, réassort fournisseur, recalcul note).

3. Communications entre services

Source	Cible	Type	Protocole / Événement	Description
Clients (Mobile/Web)	API Gateway	Sync	HTTP/2 / REST	Point d'entrée unique, terminaison TLS, validation JWT.
API Gateway	Microservices	Sync	HTTP REST interne	Routage inverse vers les services cibles selon l'URI.
Chatbot IA	MCP Gateway	Sync	JSON-RPC / REST	Traduction du langage naturel en requêtes microservices.
Order	User	Sync	GET /api/users/{id}	Contrôle bloquant de l'existence client et adresse.
Order	Cart	Sync	GET /api/cart/{id}	Récupération synchrone du snapshot panier à figer.
Cart	Catalog	Sync	GET /api/products/{id}	Contrôle du stock disponible et du prix unitaire.
Order	Catalog	Sync	POST /products/reserve	Verrouillage atomique des stocks avant confirmation.
Order	Payment	Sync	POST /payments/init	Création de la session d'encaissement et lien Stripe.
Payment	Stripe	Sync	HTTPS / REST	Appel synchrone à Stripe pour initialiser le Checkout.
Review	Catalog	Sync	GET /api/products/{id}	Vérification de l'existence du meuble noté.
Stripe	Payment	Async	Webhook checkout.session	Signal bancaire externe reçu hors session utilisateur.
Payment	Order	Async	Event PaymentConfirmed	Notification de règlement : commande passe à PAID.
Payment	Billing	Async	Event OrderPaidEvent	Déclenchement de la facture légale sans bloquer l'achat.
Billing	Notification	Async	Event InvoiceGenerated	Envoi de l'URL du PDF de facture pour transmission client.
Order	Notification	Async	Event OrderStatusChanged	Notification du statut client (expédiée, livrée).
Payment	Notification	Async	Event PaymentSucceeded	Envoi immédiat de l'email de confirmation et reçu.
Catalog	Notification	Async	Event StockBelowThreshold	Alerte gestionnaire si stock sous le seuil critique.
Catalog	Supplier	Async	Event RestockNeededEvent	Demande automatique de commande de réapprovisionnement.
Supplier	Catalog	Async	Event StockReceivedEvent	Augmentation des stocks à la réception des meubles.
Review	Supplier	Async	Event RatingAggregated	Recalcul de la note moyenne du fabricant associé.
Batches	Order / Catalog	Async	Jobs Spark SQL	Traitements nocturnes pour KPIs et modèles ALS.

4. Points d'entrée et données (1/2)

4.1. Gateways

- **Points d'entrée :**
 - ANY /api/v1/** : Routage dynamique vers microservices, terminaison TLS, validation token JWT.
 - POST /mcp/v1/query : Requête contextuelle Chatbot IA (Model Context Protocol).
- **Données :** *Stateless*. Cache Redis pour la révocation des tokens JWT et le rate limiting.

4.2. User Service

- **Points d'entrée :**
 - POST /api/users/register : Inscription nouvel utilisateur.
 - POST /api/users/login : Authentification et génération du token JWT.
 - GET | PUT /api/users/{id} : Consultation et mise à jour du profil utilisateur.
 - GET | POST /api/users/{id}/addresses : Gestion des adresses de facturation et de livraison.
- **Modèle PostgreSQL :**
 - **users** : id (UUID, PK), email (VARCHAR, UK), username (VARCHAR, UK), password_hash, first_name, last_name, phone, role (role_enum), currency (currency_enum), stripe_customer_id (UK), is_active (BOOL).
 - **addresses** : id (UUID, PK), user_id (UUID, FK users), type (address_type), street, postal_code, city, country, is_default (BOOL).

4.3. Catalog Service

- **Points d'entrée :**
 - GET /api/products : Liste paginée des meubles avec filtres (catégorie, matière, couleur, prix).
 - GET /api/products/{id} : Fiche détaillée d'un meuble (dimensions, stock disponible).
 - POST /api/products : Création / mise à jour produit (réservé Backoffice).
 - POST /api/products/reserve : Réservation atomique de quantité lors de la commande.
- **Modèle PostgreSQL :**
 - **products** : id (UUID, PK), nom (VARCHAR), prix (NUMERIC), categorie (category_enum), type_matiere (material_enum), couleur (VARCHAR), largeur/hauteur/prof. (INT), stock (INT), seuil_alerte (INT), images (TEXT[]), fournisseur_id (UUID, FK), est_actif (BOOL).

4.4. Cart Service

- **Points d'entrée :**
 - GET /api/cart/{userId} : Récupération du panier actif et calcul des sous-totaux.
 - POST /api/cart/items : Ajout d'un article dans le panier.
 - PUT /api/cart/items/{id} : Modification de quantité d'un article.
 - DELETE /api/cart/items/{id} : Suppression d'un article ; DELETE /api/cart/{id} : Vidage du panier.
- **Modèle PostgreSQL :**
 - **carts** : id (UUID, PK), user_id (UUID, FK-UQ users), code_promo (VARCHAR).
 - **cart_items** : id (UUID, PK), cart_id (UUID, FK carts), product_id (UUID, FK products), quantite (INT).

4.5. Order Service

- **Points d'entrée :**
 - POST /api/orders : Création de commande depuis le panier, validation adresse et réservation stock.
 - GET /api/orders/{id} : Détail d'une commande (lignes, montants, adresses, statuts).
 - GET /api/orders/user/{userId} : Historique complet des commandes de l'acheteur.
 - PUT /api/orders/{id}/status : Mise à jour du statut (PENDING, PAID, SHIPPED, DELIVERED).
- **Modèle PostgreSQL :**
 - **orders** : id (UUID, PK), order_number (VARCHAR, UK), user_id (UUID, FK), statut (order_status_enum), statut_paiement (payment_enum), stripe_payment_intent_id (VARCHAR, UK), shipping_address_id (UUID, FK), billing_address_id (UUID, FK), montant_total (NUMERIC), devise (currency_enum).
 - **order_items** : id (UUID, PK), order_id (UUID, FK orders), product_id (UUID, FK products), nom_produit (VARCHAR), quantite (INT), prix_unitaire (NUMERIC), total_ligne (NUMERIC).

5. Points d'entrée et données (2/2)

5.1. Payment Service

- **Points d'entrée :**
 - POST /api/payments/init : Création de la session Stripe Checkout et renvoi de l'URL de paiement.
 - POST /api/payments/webhook : Réception asynchrone des événements de paiement Stripe.
 - GET /api/payments/order/{orderId} : Consultation de l'état de la transaction financière.
- **Modèle PostgreSQL :**
 - **payments** : id (UUID, PK), order_id (UUID, UK), stripe_session_id, stripe_payment_intent_id, amount (NUMERIC), currency, status (payment_enum), paid_at (TIMESTAMPTZ).

5.2. Billing Service

- **Rôle :** Consommateur asynchrone de l'événement OrderPaidEvent. Attribue un numéro fiscal chronologique unique, calcule la ventilation HT / TVA / TTC, et compile le document légal PDF stocké sur S3 / MinIO.
- **Points d'entrée :**
 - GET /api/invoices/order/{orderId} : Récupération des métadonnées et URL du PDF de facture.
 - GET /api/invoices/{id}/pdf : Téléchargement direct du document PDF certifié.
 - GET /api/invoices/user/{userId} : Liste des factures de l'acheteur pour consultation comptable.
- **Modèle PostgreSQL :**
 - **invoices** : id (UUID, PK), numero_facture (VARCHAR, UK, ex : FACT-2026-0001), order_id (UUID, FK-UQ orders), user_id (UUID, FK users), montant_ht (NUMERIC), taux_tva (NUMERIC), montant_tva (NUMERIC), montant_ttc (NUMERIC), url_pdf (VARCHAR), date_facturation (TIMESTAMPTZ).

5.3. Notification Service

- **Points d'entrée :**
 - POST /api/notifications/send : Envoi direct déclenché par les services internes.
 - GET /api/notifications/user/{userId} : Centre de notifications in-app pour l'application mobile.
 - PUT /api/notifications/{id}/read : Marquage de lecture d'une notification.
- **Modèle PostgreSQL :**
 - **notifications** : id (UUID, PK), user_id (UUID, FK users), titre (VARCHAR), type (notif_type_enum : EMAIL, PUSH), est_lu (BOOL).

5.4. Supplier Service

- **Points d'entrée :**
 - GET /api/suppliers : Annuaire des fabricants partenaires et coordonnées.
 - POST /api/suppliers/orders : Émission d'un ordre d'achat automatique ou manuel suite à alerte stock.
 - PUT /api/suppliers/orders/{id}/receive : Réception de livraison et déclenchement d'incrément de stock.
- **Modèle PostgreSQL :**
 - **fournisseurs** : id (UUID, PK), nom (VARCHAR), email (VARCHAR), telephone (VARCHAR), adresse (TEXT).
 - **supply_orders** : id (UUID, PK), fournisseur_id (UUID, FK), statut (supply_status_enum), montant_total (NUMERIC), date_emission (TIMESTAMPTZ), date_reception (TIMESTAMPTZ).
 - **supply_order_items** : id (UUID, PK), supply_order_id (UUID, FK), product_id (UUID, FK), quantite (INT), prix_unitaire (NUMERIC).

5.5. Review Service et Batches Spark

- **Review Service (Avis) :**
 - **Points d'entrée :** POST /api/reviews (dépôt avis vérifié), GET /api/reviews/product/{id} (avis du produit).
 - **Modèle PostgreSQL (reviews) :** id (UUID, PK), user_id (UUID, FK), product_id (UUID, FK), titre (VARCHAR), note (INT 1-5), reponse_commerçant (TEXT).
- **Batches Spark (Analytique & Recommandation) :**
 - **Traitements :** Traitements Spark SQL distribués de nuit (chiffre d'affaires, panier moyen, volumes par catégorie) et entraînement du modèle de recommandation collaborative ALS (Alternating Least Squares).
 - **Modèle PostgreSQL :**
 - **product_recommendations** : user_id, product_id, score (FLOAT).
 - **kpis_ventes_journalieres** : date, nb_commandes, chiffre_affaires_ht.